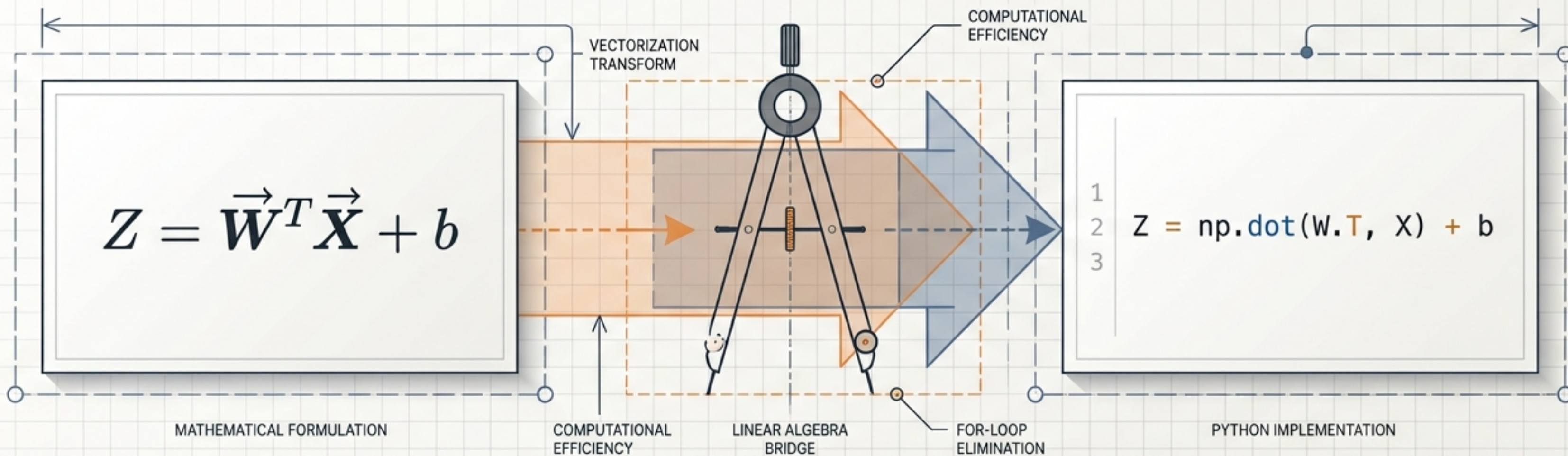


설계자의 청사진: 로지스틱 회귀의 벡터화와 파이썬 실전 최적화

For-loop 제거부터 브로드캐스팅, 비용 함수의 수학적 기원까지 (C1W2L11 - L18 완벽 매뉴얼)



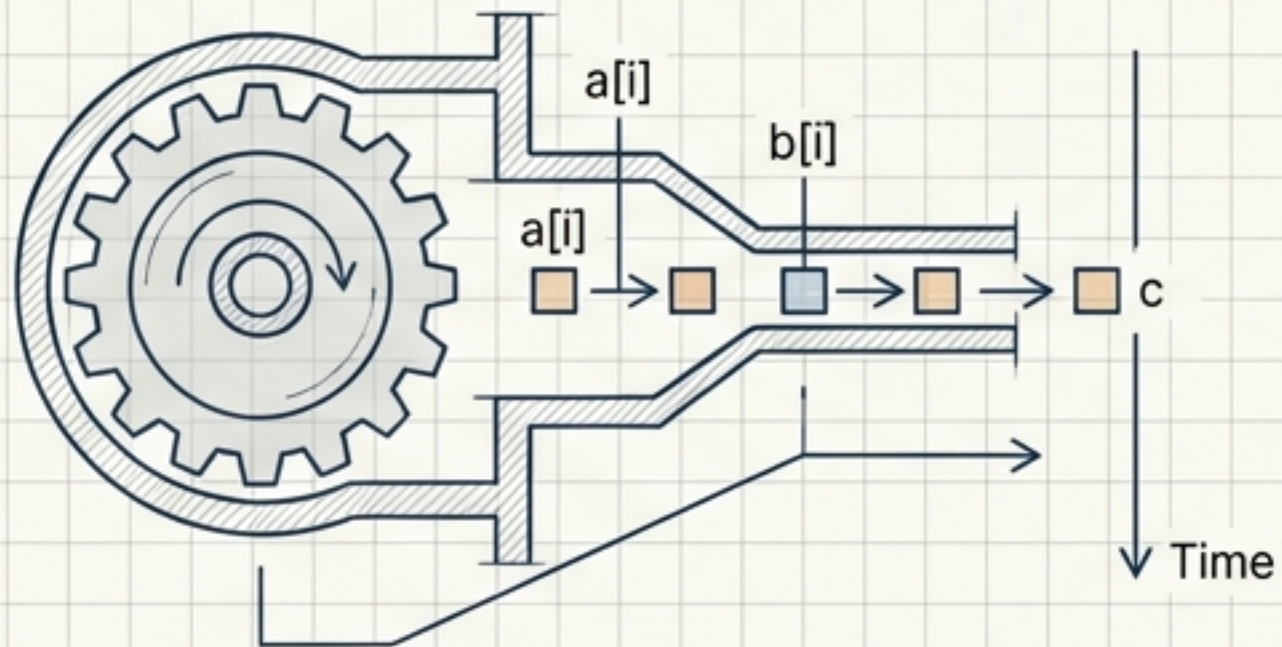
왜 벡터화인가? 300배 속도 차이의 비밀

For-loop 기반 (비벡터화)

```
c = 0
for i in range(1000000):
    c += a[i] * b[i]
```

실행 시간: 약 481ms

1분 걸릴 학습이 5시간이 걸리는 병목 현상 발생.

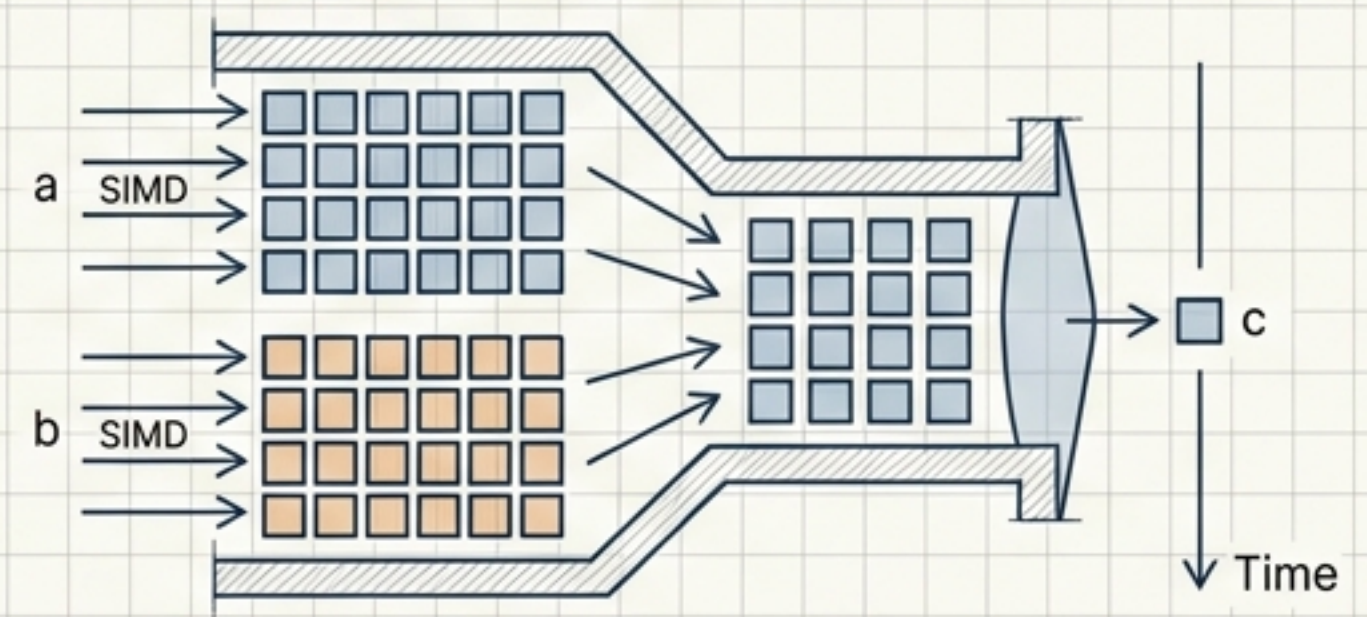


NumPy 내장 함수 (벡터화)

```
c = np.dot(a, b)
```

실행 시간: 약 1.5ms

GPU와 CPU가 제공하는 SIMD(Single Instruction Multiple Data) 기술을 통해 한 번의 명령으로 배열 전체를 동시 연산.



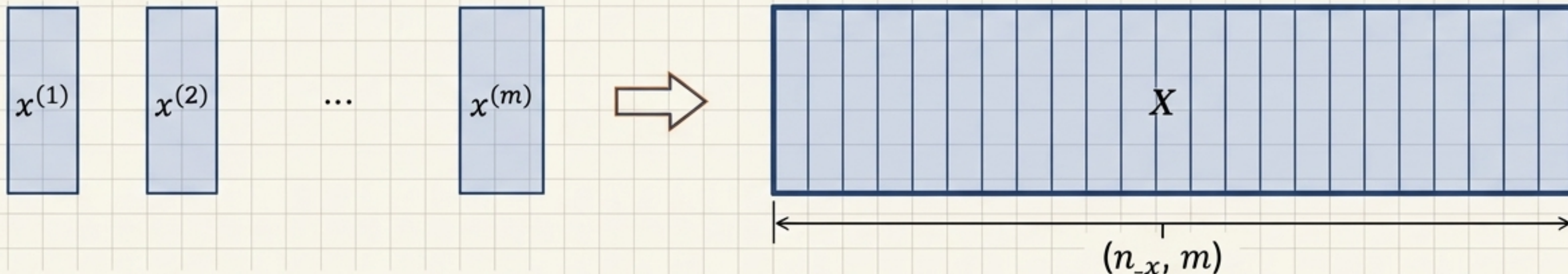
수식을 코드로 바꾸는 NumPy 내장 함수 치트시트

수학적 표기	변환	NumPy 코드	설명
$e^{v_1} \dots e^{v_n}$	➡	<code>u = np.exp(v)</code>	지수 함수 (Exponential)
$\log(v)$	➡	<code>np.log(v)</code>	로그 함수 (Logarithm)
$ v $	➡	<code>np.abs(v)</code>	절댓값 (Absolute Value)
$\max(v, 0)$	➡	<code>np.maximum(v, 0)</code>	최댓값 (음수를 0으로 자를 때 유용)
v^2	➡	<code>v**2</code>	제곱 (Square)
$1 / v$	➡	<code>1 / v</code>	역수 (Reciprocal)

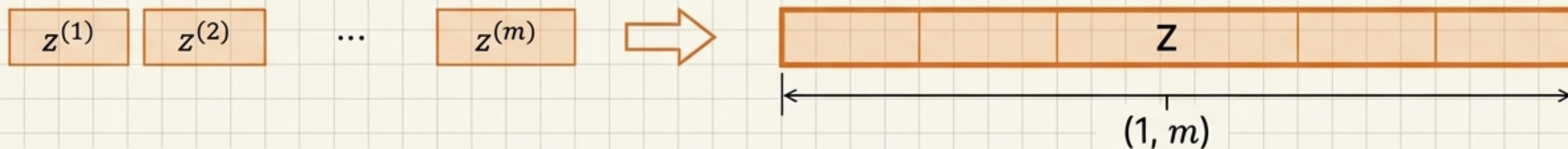
핵심 원칙: 필요한 값을 계산할 때 내장 함수를 쓸 수 있다면, For문보다 무조건 빠릅니다.

데이터의 차원 확장: 벡터(Vector)에서 행렬(Matrix)로

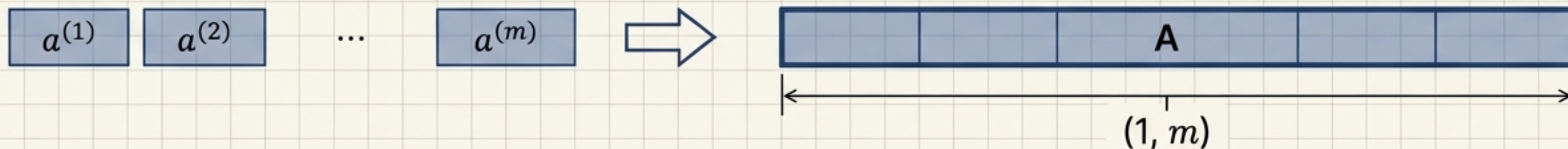
1. 입력 데이터 X (Features)



2. 선형 결합 Z (Linear Combination)



3. 활성화 값 A (Activations)



m 개의 훈련 샘플을 한 번에 계산하기 위해, 모든 개별 데이터를 열(Column) 방향으로 쌓아 거대한 행렬을 만듭니다.

정방향 전파(Forward Propagation)의 벡터화

수학적 원리

$$Z = [\mathbf{w}^T \mathbf{x}^{(1)} + b, \dots, \mathbf{w}^T \mathbf{x}^{(m)} + b]$$



$$Z = \mathbf{w}^T \mathbf{X} + b$$

- $\mathbf{w}^T : (1, n_x)$ 행 벡터
- $\mathbf{X} : (n_x, m)$ 행렬
- 결괏값 $\mathbf{Z} : (1, m)$ 행렬

파이썬 구현

```
Z = np.dot(w.T, X) + b  
A = sigmoid(Z)
```

브로드캐스팅(Broadcasting)의 마법

실수 b 는 단일 숫자이지만, 파이썬 내부에서 자동으로 $(1, m)$ 행 벡터로 확장되어 모든 m 개의 샘플에 동시에 더해집니다.

역방향 전파(Backward Prop) 도함수의 벡터화 (1)

Step 1: 오차 행렬 dZ 정의

개별 오차: $dz^{(i)} = a^{(i)} - y^{(i)}$

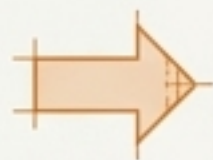
행렬화: $dZ = [dz^{(1)}, dz^{(2)} \dots dz^{(m)}]$

$$dZ = A - Y \quad (\text{단 한 번의 행렬 뺄셈 연산})$$



Step 2: 편향의 기울기 db 도출

$$db = \frac{1}{m} \sum_{i=1}^m dz^{(i)}$$

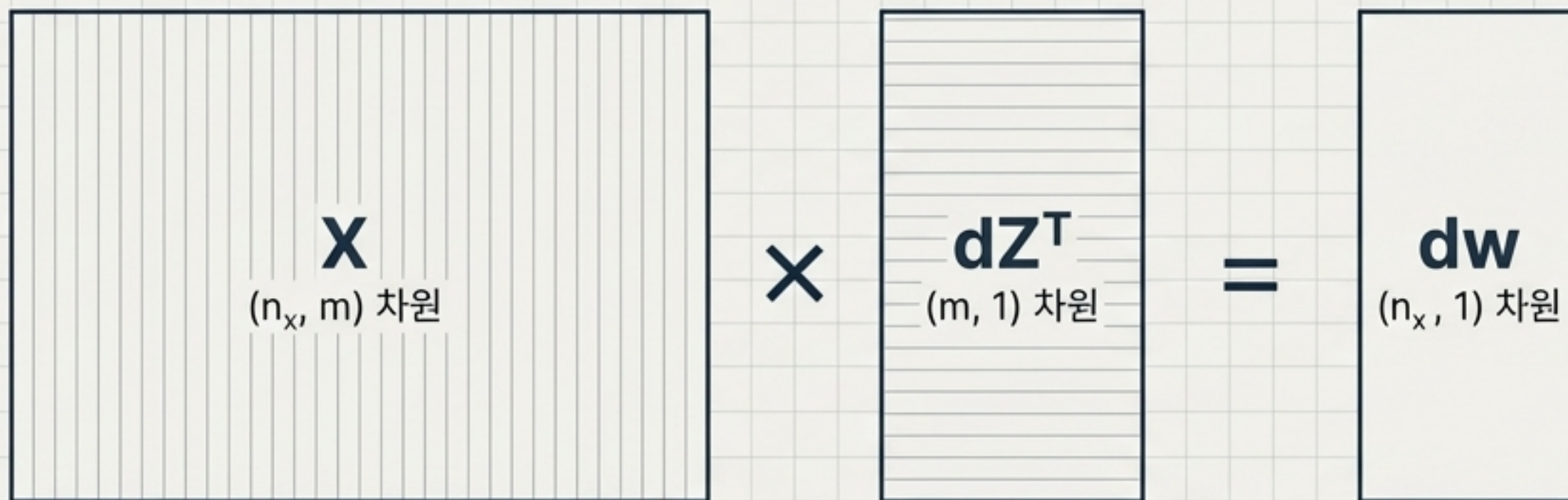


$$db = (1/m) * np.sum(dZ)$$

dZ 행렬의 모든 요소를 더하는 함수 `np.sum()`을 통해 For문을 완벽히 대체합니다.

Dimension Check

$$dw = \frac{1}{m} X \cdot dZ^T$$



$$dw = (1/m) * np.dot(X, dZ.T)$$

X와 **dZ^T**를 내적(np.dot)하면, 내부적으로 $\mathbf{x}^{(i)} \cdot \mathbf{dz}^{(i)}$ 의 합이 자동으로 계산되어
가중치 **w**와 동일한 차원의 열 벡터가 완성됩니다.

Before (비효율적인 For-loop 구현)

```
J = 0, dw1 = 0, dw2 = 0, db = 0
for i = 1 to m:
    z[i] = w.T * x[i] + b
    a[i] = sigmoid(z[i])
    J += -[y[i]log(a[i]) +
          (1-y[i])log(1-a[i])]
    dz[i] = a[i] - y[i]
    for j = 1 to n_x:
        dw[j] += x[i][j] * dz[i]
    db += dz[i]
```

수십 줄의 느리고 복잡한 중첩 반복문

After (단 5줄의 완벽한 벡터화)

```
# m개의 샘플을 for문 없이 동시 처리
Z = np.dot(w.T, X) + b
A = sigmoid(Z)
dZ = A - Y
dw = (1/m) * np.dot(X, dZ.T)
db = (1/m) * np.sum(dZ)

# 매개변수 갱신
w = w - learning_rate * dw
b = b - learning_rate * db
```

단일 반복(Iteration) 내 For-loop 완전 제거!
거대한 훈련 세트 전체를 동시에 처리합니다.

Matrix A (3x4)

	사과	소고기	달걀	감자
탄수화물	56	0	4.4	68
단백질	1.2	104	52	8
지방	1.8	135	99	0.9

Step 2: 행렬 전체를 벡터로 나누기

	사과	소고기	달걀	감자
탄수화물	56	0	4.4	68
단백질	1.2	104	52	8
지방	1.8	135	99	0.9

÷

Step 1: 열(Column) 합계 구하기

```
cal = A.sum(axis=0)
```

세로축 합계를 구해 (1, 4) 형태의 배열 생성.
(예: 사과 총합 59)

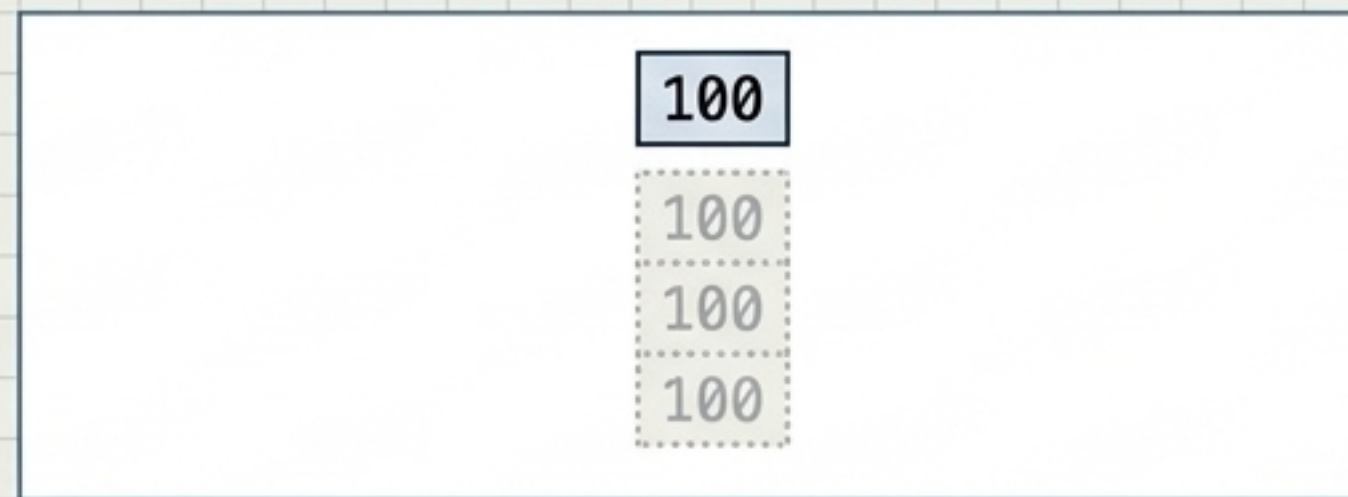
```
percentage = 100 * A / cal.reshape(1,4)
```

사과	소고기	달걀	감자
[59	239	155.4	768]
[59	239	155.4	76.9]

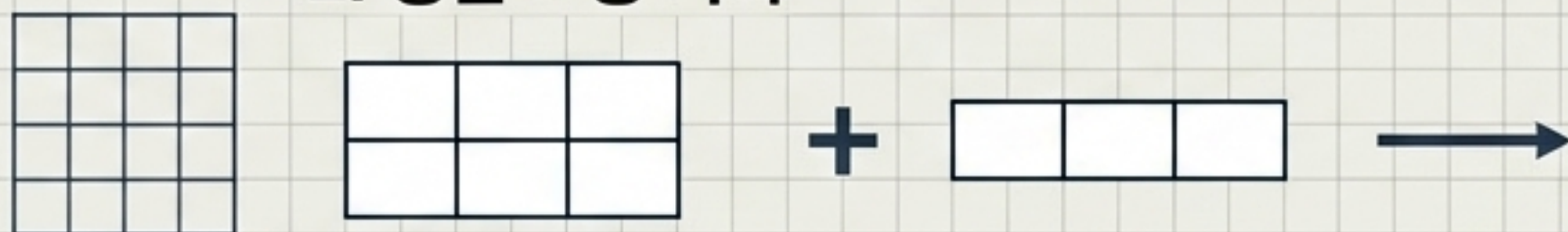
/ Stretching

[59	239	155.4	76.9]
-----	-----	-------	-------

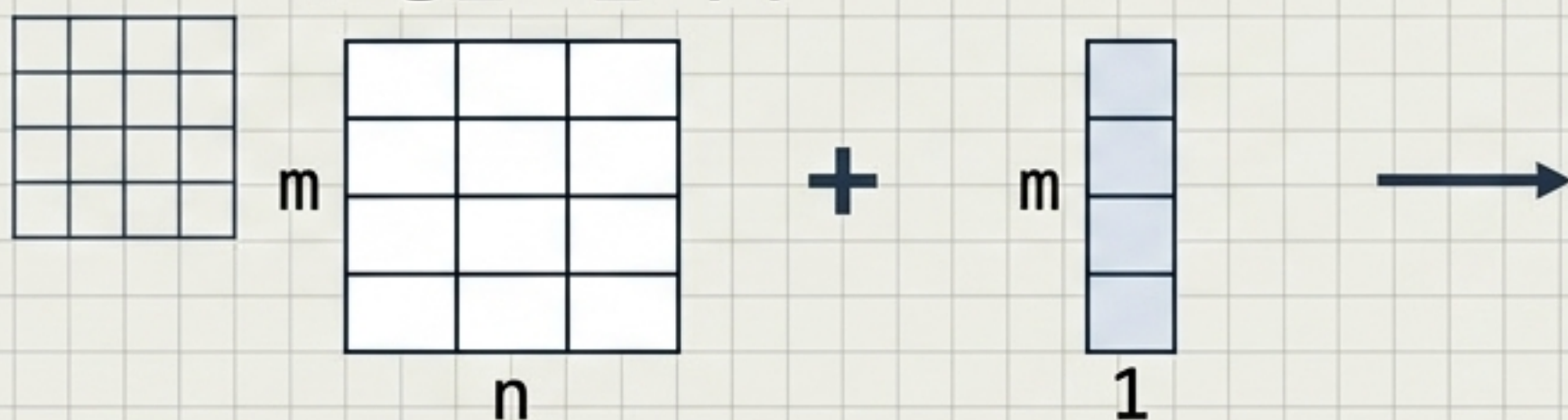
1. 열 벡터 + 스칼라 (상수)



2. 행렬 + 행 벡터



3. 행렬 + 열 벡터



핵심 원칙: 차원이 부족한 배열이 더 큰 배열의 형태(Shape)에 맞게 자동으로 복사(Stretching)된 후 요소별(Element-wise) 연산을 수행합니다.

Rank-1 Array (사용 금지)

- 생성: `a = np.random.randn(5)`
- 형태: `a.shape -> (5,)`
- 특징:
행 벡터도 열 벡터도 아닌 모호한 구조.
`a.T`(전치)를 해도 `a`와 똑같이 생김.
- 치명적 오류:
`np.dot(a, a.T)`의 결과가 행렬 외적이
아니라 단일 수(Scalar)로 도출됨.
선형대수학적 직관성 파괴!

명시적 열 벡터 (권장)

- 생성: `a = np.random.randn(5, 1)`
- 형태: `a.shape -> (5, 1)`
- 특징:
`a.T` 결과가 대괄호 2개인 `[[...]]`
형태의 (1, 5) 행 벡터로 정상 변환됨.
- 정상 동작:
`np.dot(a, a.T)` 연산 시 예측대로
5x5 외적(Outer Product) 행렬 도출.

Checklist

1. 배열 생성 시 명시적 차원 지정

```
a = np.random.randn(5, 1)
```

항상 괄호 안에 두 개의 숫자를 넣어
행렬(Matrix) 형태로 차원을 지정하세요.

2. Assert 문 적극 활용 (방어적 프로그래밍)

```
assert(a.shape == (5, 1))
```

매우 저렴한 연산이므로 코드 곳곳에 삽입해
차원을 검증하고 문서화 역할을 부여하세요.

2. Assert 문 적극 활용 (방어적 프로그래밍)

```
assert(a.shape == (5, 1))
```

매우 저렴한 연산이므로 코드 곳곳에
삽입해 차원을 검증하고 문서화 역할을
부여하세요.

3. Reshape의 주저 없는 사용

```
a = a.reshape((5, 1))
```

차원이 모호한 랭크-1 배열을 얻게 되면
즉시 reshape로 차원을 고정하세요.
(실행 시간 상수 $O(1)$ 로 성능 저하 없음)

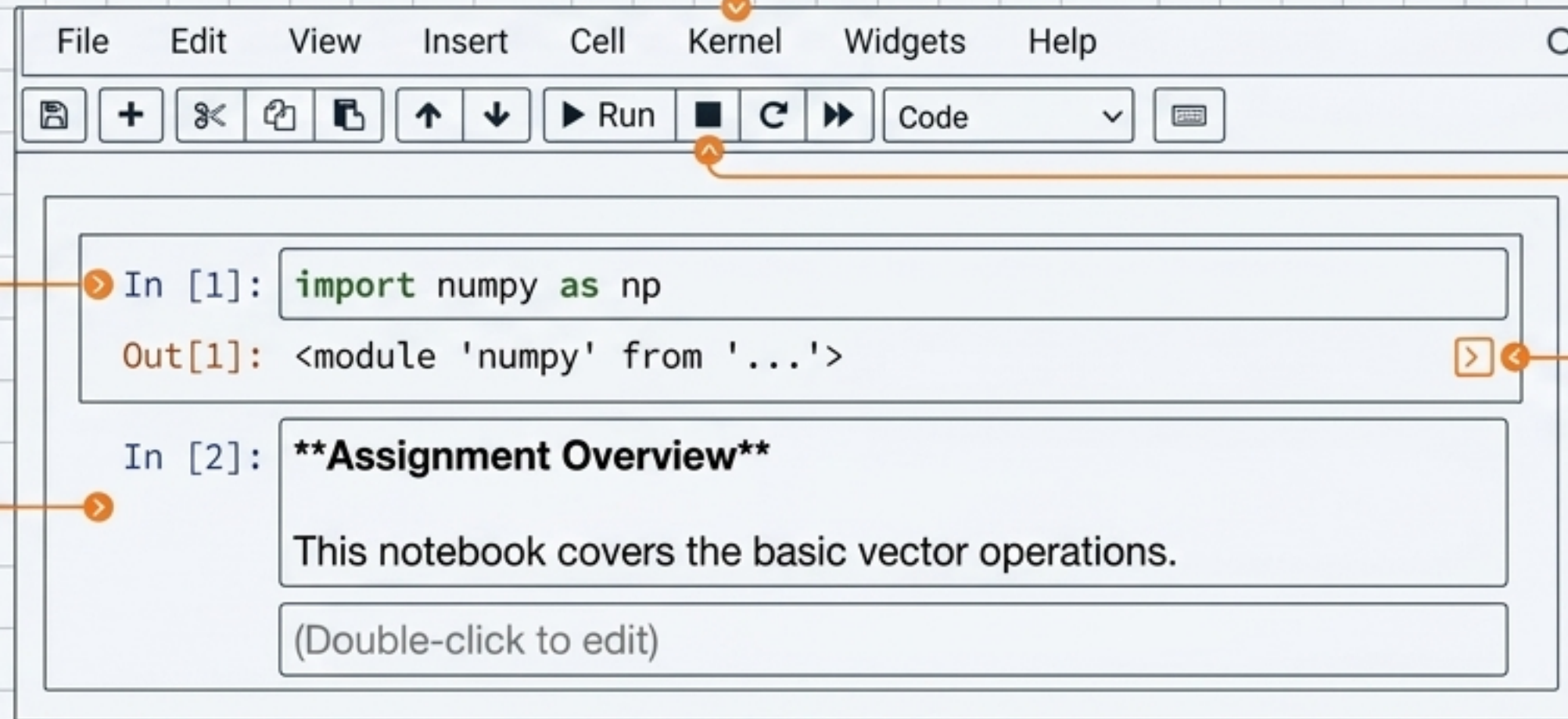
Jupyter Notebook 필수 가이드 (과제용)

4. 초기화 코드 필수 실행

작성할 코드가 없더라도 최상단의 Import 셀은 반드시 실행하고 넘어가야 합니다.

2. 커널 재시작 (Kernel > Restart)

연결이 끊기거나 변수 상태가 꼬였을 때, 모든 메모리를 초기화하는 마법의 버튼.



1. 셀 실행 (Shift + Enter)

현재 셀을 서버 커널에서 실행하고 다음 셀로 이동합니다.

3. 마크다운(Markdown) 셀

더블클릭 시 텍스트 편집이 가능하며, 실행 시 깔끔한 문서로 렌더링됩니다.

1. 문제의 출발점 (이진 분류)

$$y = 1 \text{ 일 때, } p(y|\mathbf{x}) = \hat{y}$$

$$y = 0 \text{ 일 때, } p(y|x) = 1 - \hat{y}$$

2. 위 두 식을 단 하나의 수식(Equation)으로 병합

$$p(y|\mathbf{x}) = \hat{y}^y(1 - \hat{y})^{(1-y)}$$

3. 수학적 증명 (0승 소거법)

만약 $y = 1$ 이라면?

$$\hat{y}^1 \cdot (1 - \hat{y})^0$$

어떤 수의 0승은
1로 소거됨

결과: $\hat{y}$ (정확히 일치!)

만약 $y = 0$ 이라면?

~~$\hat{y}^0 \cdot (1 - \hat{y})^1$~~

1로 소거됨

결과: $1 - \hat{y}$ (정확히 일치!)

최대 우도 추정(MLE)과 로그 손실 함수(Log Loss)의 탄생

1. IID 가정과 확률의 곱

$$\prod_{i=1}^m p(y^{(i)} | x^{(i)})$$

전체 데이터의 확률은
독립 샘플들의 개별 확률의
곱(Product)과 같습니다.

2. 로그(Log) 적용

$$\sum_{i=1}^m \log p(y^{(i)} | x^{(i)})$$

단조 증가함수인 Log를 씌워
곱셈을 덧셈으로 변환.
 $y \log \hat{y} + (1 - y) \log(1 - \hat{y})$
단일 손실 형태 도출.

3. 가능도 최대화 = 비용 최소화

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

통계학의 확률 '최대화'를 딥러닝의
비용 '최소화'로 바꾸기 위해 음수
부호(-)와 $1/m$ 스케일링을
추가합니다.

**결론: 딥러닝에서 비용 함수를 최소화하는 것은 훈련 데이터의 확률(가능도)을
최대화하는 통계학적 과정과 완벽히 동일합니다.**